Rapid Fire: Traps, HR & Last-Minute Drill

Output prediction, behavioural questions, one-minute answers

5 topics · 31 flashcards · 20 quiz questions · 13 code examples

How to use this in 4 days

1. **Answer the quiz before you read.** Getting one wrong makes the answer stick; reading twice does not.
2. **Say it out loud.** Every topic ends with a scripted interview answer in the orange box. Reading it silently is not studying.
3. **Type the code.** Your hands remember what your eyes skim past.
4. **Days 1 and 2 are non-negotiable.** OOP, Collections and exceptions are where fresher rounds are won.
5. **Rehearse the project pitch out loud, three times.** It is the highest-leverage question in a fresher interview.

Rapid Fire: Rapid Fire: Traps, HR & Last-Minute Drill

Output prediction, behavioural questions, one-minute answers

5 topics · 31 flashcards · 20 quiz questions · ~2.1 hours

> **Goal.** The night before and the morning of: run the output traps until you get them all right, read your one-minute answers out loud, and walk in with your HR stories ready.

Schedule

- T-1 evening (1h): Output traps — aim for 90%+
- T-1 evening (45m): One-minute answers out loud, without notes
- T-1 evening (30m): Write your HR stories in STAR format
- Morning of (30m): Re-run the traps, skim the cheat sheet
- 10 min before: Breathe. You know more than you think.

1. Output Prediction Traps (Part 1: Strings & Numbers) Trap 25 min

These are the questions where the interviewer writes five lines on the board and asks "what does this print?" They're testing whether you actually understand the memory model, not whether you've memorised syntax. Run the quiz until you're perfect.

Key points

- String literal pooling: `==` compares references, `equals()` compares contents.
- Compile-time constant folding happens for literals, not for variables.
- Integer cache: -128 to 127 is cached; beyond that every boxing is a new object.
- Integer division truncates; floating-point division by zero gives Infinity, not an exception.
- `+` with a String anywhere converts the whole expression to concatenation, left to right.

Code

Six traps in one go — cover the output and check

```
// 1
String a = "hi", b = "hi";
System.out.println(a == b);                     // true  (pool)

// 2
String c = new String("hi");
System.out.println(a == c);                     // false (new object)

// 3
String d = "h" + "i";
System.out.println(a == d);                     // true  (constant folding)

// 4
String e = "h"; String f = e + "i";
System.out.println(a == f);                     // false (runtime concat)

// 5
Integer x = 100, y = 100;
Integer p = 200, q = 200;
System.out.println(x == y);                     // true  (cache)
System.out.println(p == q);                     // false (outside cache)

// 6
System.out.println(10 / 3);                     // 3
System.out.println(10 / 3.0);                   // 3.3333333333333335
System.out.println(1.0 / 0);                    // Infinity
System.out.println("5" + 2 + 3);                // "523"
System.out.println(5 + 2 + "3");                // "73"
```

```
Output:
true
false
true
false
true
false
3
3.3333333333333335
Infinity
523
73
```

Trap: null concatenation and StringBuilder equality

```
String s = null;
System.out.println(s + "!");          // "null!" - no NPE!
// s.concat("!");                     // NullPointerException
// s.length();                        // NullPointerException

StringBuilder sb1 = new StringBuilder("hi");
StringBuilder sb2 = new StringBuilder("hi");
System.out.println(sb1 == sb2);       // false
System.out.println(sb1.equals(sb2));  // false - NOT overridden!
System.out.println(sb1.toString().equals(sb2.toString()));  // true
```

Output:

```
null!
false
false
true
```

SAY IT LIKE THIS IN THE INTERVIEW

"String literals are interned in a pool and reused; `new String()` always creates a fresh object. Compile-time constants get folded, but concatenation involving a variable happens at runtime. Wrappers cache -128 to 127, so == works there and fails beyond it — which is why I always use equals()."

Flashcards

Q. `String a="hi"; String b=new String("hi"); a==b`?
A. false — new String() creates a separate heap object instead of reusing the pooled literal.

Q. `Integer a=127,b=127; a==b` and `Integer c=128,d=128; c==d`?
A. true then false — the Integer cache covers -128..127 only.

Q. `"5"+2+3` vs `5+2+"3"`?
A. "523" and "73" — evaluation is left to right and the first String encountered converts the rest.

Q. Does `null + "!"` throw NPE?
A. No — String concatenation converts null to the text "null". Calling a method on null throws NPE.

Q. Do StringBuilder and StringBuffer override equals()?
A. No — their equals() is reference equality. Convert to String first to compare contents.

Quiz

Q1. What does this print? `String s1="Java"; String s2="Ja".concat("va"); String s3=s1.intern(); System.out.println(s1==s2); System.out.println(s1==s3);`

[ ] A. true true
[x] B. false true
[ ] C. false false
[ ] D. true false

Why: concat() runs at runtime and returns a new String, so s1 != s2. intern() returns the pooled instance, so s1 == s3.

Q2. `System.out.println((byte) 130);`

[ ] A. 130
[x] B. -126
[ ] C. 127
[ ] D. Compile error

Why: byte is signed 8-bit (-128..127). 130 in binary is 10000010, which as a signed byte is -126. Overflow wraps around.

Q3. What is the output? `int i=0; System.out.println(i++ + i++ + i++);`

[x] A. 3
[ ] B. 6

[ ] C. 0
[ ] D. 1

> **Why:** 0 + 1 + 2 = 3. Each i++ yields the current value then increments: first is 0 (i→1), second is 1 (i→2), third is 2 (i→3).

Q4. `char c = 'A'; System.out.println(c + 1);`

[ ] A. B
[x] B. 66
[ ] C. A1
[ ] D. Compile error

> **Why:** char promotes to int in arithmetic: 'A' is 65, so 66. To get 'B' you'd cast: `(char)(c + 1)`.

Q5. Which of these throws NullPointerException?

[ ] A. String s = null; System.out.println(s + "");
[ ] B. String s = null; System.out.println(String.valueOf(s));
[x] C. String s = null; System.out.println(s.toString());
[ ] D. String s = null; System.out.println(s == null);

> **Why:** Calling any method on null throws NPE. Concatenation, String.valueOf() and == are all null-safe (String.valueOf returns "null").

Q6. `double d = 0.1 + 0.2; System.out.println(d == 0.3);

[ ] A. true
[x] B. false
[ ] C. Compile error
[ ] D. Throws ArithmeticException

> **Why:** 0.1 + 0.2 = 0.30000000000000004 in binary floating point. Never compare doubles with == — use a tolerance (epsilon) or BigDecimal for money.

2. Output Prediction Traps (Part 2: OOP & Exceptions) Trap 25 min

Part two covers method dispatch, initialisation order, static vs instance, and the finally block. These separate people who've written Java from people who understand it.

Key points

- Instance methods dispatch at runtime on the object; static methods and fields resolve at compile time on the reference type.
- Fields are never polymorphic — field access uses the declared type.
- Initialisation: static (once) → instance blocks → constructor, parent before child.
- `finally` always runs — and a `return` inside it overrides the try's return.
- The return value is captured before `finally` executes, so mutating the variable there changes nothing.

Code

Dispatch and hiding

```
class A {
   int v = 1;
   void show()     { System.out.println("A.show"); }
   static void st(){ System.out.println("A.static"); }
}
class B extends A {
   int v = 2;
   @Override void show() { System.out.println("B.show"); }
   static void st()       { System.out.println("B.static"); }
}

A ref = new B();
ref.show();            // B.show   <- runtime dispatch (overriding)
ref.st();              // A.static <- compile-time (static HIDING)
System.out.println(ref.v);   // 1   <- fields are NOT polymorphic

// Cast to call the parent's version:
((A) new B()).show();  // B.show - a cast doesn't change runtime dispatch!
```

```
Output:
B.show
A.static
1
B.show
```

Initialisation order, end to end

```
class Parent {
   static { System.out.println("P static"); }
   { System.out.println("P instance"); }
   Parent() { System.out.println("P ctor"); }
}
class Child extends Parent {
   static { System.out.println("C static"); }
   { System.out.println("C instance"); }
   Child() { System.out.println("C ctor"); }
}
public class Main {
   public static void main(String[] a) {
       System.out.println("--- first ---");
       new Child();
       System.out.println("--- second ---");
       new Child();
   }
}
// Static blocks run ONCE, parent before child, at first class load.
// Then per object: P instance -> P ctor -> C instance -> C ctor.
```

Output:

```
--- first ---
P static
C static
P instance
P ctor
C instance
C ctor
--- second ---
P instance
P ctor
C instance
C ctor
```

finally and return — the three cases

```
static int a() { try { return 1; } finally { return 2; } }
static int b() { int x = 1; try { return x; } finally { x = 2; } }
static int c() {
   try { return 1 / 0; }
   catch (Exception e) { return 3; }
   finally { System.out.print("finally "); }
}
System.out.println(a());   // 2  <- finally's return wins
System.out.println(b());   // 1  <- value snapshotted before finally
System.out.println(c());   // "finally 3" - finally runs before the return lands
```

Output:

```
2
1
finally 3
```

Trap: constructor calls an overridable method

```
class Base {
   Base() { print(); }               // calls an OVERRIDABLE method!
   void print() { System.out.println("Base"); }
}
class Derived extends Base {
   private String name = "derived";   // initialised AFTER super()
   @Override void print() { System.out.println("name = " + name); }
}
new Derived();   // "name = null" - the subclass field isn't set yet!

// LESSON: never call overridable methods from a constructor.
// The subclass's fields are still at their default values.
```

```
Output:
name = null
```

SAY IT LIKE THIS IN THE INTERVIEW

"Instance methods are dispatched at runtime by the object's type, but static methods are hidden and resolved at compile time by the reference type — and fields are never polymorphic at all. Initialisation runs static blocks once, parent before child, then instance blocks and constructors per object. And finally always runs; a return in finally overrides the try's return, though the returned value is snapshotted before finally executes."

Flashcards

Q. Are fields polymorphic in Java?
A. No. Only instance methods use dynamic dispatch. Field access is resolved by the declared (reference) type at compile time.

Q. What is method hiding vs overriding?
A. Static methods are hidden (resolved by reference type at compile time); instance methods are overridden (resolved by object type at runtime).

Q. Does a cast change which overridden method runs?
A. No — casting only changes the compile-time view. The JVM still dispatches to the actual object's method.

Q. Full initialisation order for `new Child()` where Child extends Parent?
A. Parent static → Child static → Parent instance block → Parent constructor → Child instance block → Child constructor. Statics run only once per class load.

Q. Why shouldn't a constructor call an overridable method?
A. The subclass's fields aren't initialised yet (super() runs first), so the overridden method sees default values — a classic source of NPEs.

Q. What does `try { return 1; } finally { return 2; }` return?
A. 2. A return in finally overrides the try's return and swallows any exception — which is why it's an anti-pattern.

Quiz

Q1. What does this print? `class A{ int x=1; void p(){System.out.print(x);} } class B extends A{ int x=2; } new B().p();`

[x] A. 1
[ ] B. 2
[ ] C. Compile error
[ ] D. 0

Why: Fields aren't polymorphic. A.p() prints A's x (1); B declares its own x which merely shadows it in B's scope.

Q2. `static int m(){ try{ throw new RuntimeException(); } finally{ return 5; } }`

[ ] A. Throws the exception
[x] B. Returns 5 — the exception is swallowed
[ ] C. Compile error
[ ] D. Returns 0

Why: A return in finally discards the in-flight exception entirely. This is exactly why returning from finally is discouraged.

Q3. What is the output order when creating a Child of a Parent for the second time?

[ ] A. All statics and instance blocks again
[x] B. Only instance blocks and constructors — statics ran once at first class load
[ ] C. Nothing
[ ] D. Only constructors

Why: Static initialisers run exactly once, when the class is first loaded. Instance initialisers and constructors run for every new object.

Q4. Why does `new Derived()` print "name = null" when the constructor calls an overridden print()?

[ ] A. The field is final
[x] B. super() runs before the subclass's field initialisers, so the field still holds its default value
[ ] C. print() is static
[ ] D. It's a compiler bug

Why: Order: allocate → zero fields → call super() (which invokes the overridden method) → then run the subclass's field initialisers and constructor body.

Q5. Which is true about calling an overridden method from a constructor?

[ ] A. It's recommended for initialisation
[x] B. It's a known anti-pattern — make such methods private, final or static
[ ] C. It's required by Java
[ ] D. It only fails with primitives

Why: Calling overridable methods from constructors leads to partially-constructed objects. Keep constructor-called methods private/final/static.

3. One-Minute Answers (Verbal Rapid Fire) Must-know 30 min

The rapid-fire round gives you about 60 seconds per question. These are the definitions you should be able to deliver without thinking — say them out loud until they're automatic.

Key points

- Answer in 2–3 sentences. Definition → key mechanism → why it matters.
- If you blank, say "I haven't used that directly" and reason from first principles — never bluff.
- Pauses are fine. Take two seconds, structure the answer, then speak.
- Volume and pace matter: slow down, and stop at the end of each answer instead of trailing off.

Code

20 answers to have on the tip of your tongue

```
1.  OOP?            Encapsulation, Inheritance, Polymorphism, Abstraction.
2.  Platform independent?  javac -> bytecode, the JVM runs it anywhere.
3.  JDK/JRE/JVM?    Dev kit / runtime / the engine that executes bytecode.
4.  == vs equals?   == compares references, equals compares contents.
5.  Why is String immutable?  Security, thread-safety, hash caching, pooling.
6.  final?          A constant variable, a non-overridable method, a non-extendable class.
7.  Overload vs override?  Compile-time vs runtime polymorphism.
8.  Abstract vs interface?  Shared state + IS-A vs a contract, multiple inheritance of type.
9.  Checked vs unchecked?  Compile-time enforced vs RuntimeExceptions from bugs.
10. Collection vs Collections?  The interface vs the utility class.
11. ArrayList vs LinkedList?  Array-backed O(1) access vs linked, fast end inserts.
12. HashMap internals?  Buckets by hash, chaining, treeify at 8, resize at 0.75.
13. Why override hashCode with equals? Equal objects must hash equally or hash collections
    break.
14. Fail-fast vs fail-safe?  Throw CME vs iterate a snapshot.
15. Generics?  Compile-time type safety implemented by type erasure.
16. PECS?  Producer extends, Consumer super.
17. Lambda?  An instance of a functional interface; captures must be effectively final.
18. Stream?  A lazy pipeline: source -> intermediate -> terminal; consumed once.
19. synchronized vs volatile?  Exclusion + visibility vs visibility only.
20. sleep vs wait?  Holds the lock vs releases it (and needs notify).
```

Full-stack rapid fire

```
21. IoC vs DI?    The principle vs the mechanism (constructor injection preferred).
22. Bean scope?   Singleton by default; also prototype, request, session.
23. Spring vs Boot?  Boot = auto-config + starters + embedded server + Actuator.
24. Auto-config?  Conditional beans (@ConditionalOnClass / @ConditionalOnMissingBean).
25. @Component/@Service/@Repository/@Controller?  Stereotypes per layer;
    @Repository adds exception translation.
26. DispatcherServlet?  Front controller that routes each request to a handler.
27. @PathVariable vs @RequestParam vs @RequestBody?  Path / query / JSON body.
28. Why DTOs?     Stable contract, no leaked fields, no JSON recursion.
29. 201 vs 204?   Created (POST) vs No Content (DELETE).
30. JPA vs Hibernate?  Spec vs implementation; Spring Data generates repositories.
31. N+1 problem?  1 query + N lazy loads; fix with JOIN FETCH / @EntityGraph.
32. Lazy vs Eager?  Load on access vs load immediately; @ManyToOne defaults EAGER.
33. @Transactional placement?  Service layer, readOnly for queries, proxy-based.
34. Dirty checking?  Hibernate auto-UPDATEs changed managed entities at flush.
35. LazyInitializationException?  Lazy access after the session closed.
36. Index trade-off?  Faster reads, slower writes, storage cost.
37. WHERE vs HAVING?  Filter rows before grouping vs groups after.
38. INNER vs LEFT JOIN?  Matching rows only vs all left rows with NULLs.
39. JWT flow?  Login -> signed token -> Bearer header -> filter validates -> SecurityContext.
40. BCrypt?  Salted, deliberately slow password hashing - never encrypt passwords.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Answer in three beats: what it is, how it works, why it matters. Two sentences, then stop and let them ask for more."

Flashcards

Q. Give a 2-sentence answer for 'what is polymorphism?'
A. Polymorphism lets the same call take many forms: overloading is resolved at compile time from argument types, overriding at runtime from the actual object type. It's what lets me write `List l = new ArrayList<>()` and swap the implementation without changing callers.

Q. Give a 2-sentence answer for 'what is dependency injection?'
A. DI means a class receives its dependencies from outside rather than creating them with `new`. I prefer constructor injection because it keeps fields final, makes dependencies explicit and enables unit testing with plain mocks.

Q. Give a 2-sentence answer for 'how does HashMap work?'
A. It's an array of buckets indexed by hash(key) & (n-1); on a collision entries are chained, and since Java 8 buckets over 8 entries are treeified to keep the worst case at O(log n). It doubles and rehashes when size exceeds 0.75 × capacity.

Q. Give a 2-sentence answer for 'what is the N+1 problem?'
A. One query fetches the parents and then one more fires per parent to lazily load each child collection. I fix it with JOIN FETCH or @EntityGraph so it becomes a single query.

Q. Give a 2-sentence answer for 'checked vs unchecked exceptions?'
A. Checked exceptions are verified at compile time — I must catch or declare them — while unchecked RuntimeExceptions usually indicate a bug I should fix. Error types like OutOfMemoryError should never be caught.

Q. How do you handle a question you don't know?
A. Say honestly that you haven't used it, then reason aloud from first principles about how you'd approach it. Interviewers score the reasoning far higher than a confident guess.

Q. Why is pausing before you answer a good thing?
A. A two-second pause signals thoughtfulness instead of panic, and it stops you from rambling into an answer you can't defend.

Quiz

Q1. What's the recommended structure for a rapid-fire answer?

[ ] A. Talk for as long as you can to show depth
[x] B. Definition → key mechanism → why it matters, in 2-3 sentences
[ ] C. Name as many related terms as possible
[ ] D. Ask the interviewer to clarify first

Why: Three beats, then stop. It shows structure and lets the interviewer pull more detail if they want it.

Q2. You freeze on a question. The best move is:

[ ] A. Guess loudly
[x] B. Pause, say you're not certain, and reason from first principles
[ ] C. Say 'next question'
[ ] D. Repeat the question back

Why: Visible reasoning is what's being assessed. A calm, structured attempt beats both silence and a bluff.

4. HR & Behavioural Questions Must-know 30 min

Technical rounds get you shortlisted; HR rounds get you the offer — or quietly cut you. The good news: there are only about ten questions, and you can prepare all of them tonight.

Key points

- Every behavioural answer should be a **story**: STAR (Situation, Task, Action, Result) with a measurable outcome.
- **Never badmouth** a previous employer, manager or college. Frame everything as a learning experience.
- For "why should we hire you?": match your skills to *their* requirements, don't recite your resume.
- For "where do you see yourself in 5 years?": show ambition that fits the role's growth path (senior dev / tech lead), not "your job".
- Have a genuine reason for wanting *this* company — their stack, their product, their scale.
- Prepare two questions to ask them (about the team's stack and engineering practices).

Code

The ten questions — with a model angle for each

```
1. "Tell me about yourself."
   Present -> Past -> Future. 90 seconds max.
   "I'm a Java full-stack developer. I trained at [institute], where I built
   an Order Management System with Spring Boot and JPA - I owned the order
   and payment modules. Before that I completed my degree in [field]. I'm
   looking for a role where I can grow on real backend systems, which is
   why this position interested me."

2. "Why should we hire you?"
   Match their needs: "You mentioned you need someone who can work across
   Spring Boot and SQL. I built [project] using exactly that, including
   fixing an N+1 query problem, so I can contribute from week one while
   picking up your domain quickly."

3. "What are your strengths?"
   Pick 2 that match the job, with evidence. "I'm persistent with debugging -
   I spent two days on a transaction rollback issue and ended up learning
   how Spring's proxy-based transactions actually work."

4. "What is your biggest weakness?"
   Real, non-fatal, with a fix in progress. "I used to under-invest in
   tests. Now I write JUnit tests alongside every service method, and my
   last project had around 70% coverage on business logic."

5. "Where do you see yourself in 5 years?"
   "Growing into a senior backend engineer - deeper on system design,
   performance and mentoring juniors. I want to be the person who owns a
   service end to end."

6. "Why do you want to work here?"
   Be specific: their product, scale, stack or engineering blog. Never
   "because it's a big company".

7. "Tell me about a conflict / difficult teammate."
   STAR. Depersonalise it, focus on communication and the outcome.

8. "Describe a time you failed."
   Real failure, ownership (no blaming), what you changed. Show the lesson.

9. "What are your salary expectations?" (usually a later round)
   Give a researched range and stay flexible: "Based on the market for
   Java developers with 0-2 years, I'm looking at around X to Y, but I'm
   open to discussing the full package."

10. "Do you have any questions for us?"
   ALWAYS have two. "What does the team's stack look like day to day?"
   "How do you handle code review and deployments?"
```

STAR template — fill in two stories tonight

```
STORY 1: A hard technical problem
 S - "Our order history endpoint took 3 seconds to load."
 T - "I needed to find and fix the bottleneck before the demo."
 A - "I enabled Hibernate SQL logging and saw 200 queries instead of one -
      a classic N+1 from lazy-loaded order items. I rewrote the query with
      JOIN FETCH and added pagination."
 R - "Response time dropped from 3 seconds to under 200ms, and I added
      SQL logging to our dev config so we'd catch it earlier next time."

STORY 2: A disagreement or failure
 S - "I disagreed with a teammate about returning entities directly from
      our controllers instead of DTOs."
 T - "We needed to decide before the API was published to the frontend
      team."
 A - "I built a quick prototype showing how bidirectional JPA relations
      caused infinite JSON recursion and leaked the password hash field."
 R - "We standardised on DTOs for every endpoint, and I wrote the mapping
      layer. The frontend contract stopped changing with our DB schema."

// Rules for these stories:
//   - Own your part ("I", not "we", when describing your action)
//   - Include a NUMBER in the result
//   - End with what you learned or changed going forward
//   - Never criticise a person - criticise the problem
```

Red flags to avoid

```
DON'T:
 x "We did everything as a team" for every answer (no ownership)
 x Badmouthing a past manager, company or college
 x "I don't have any weaknesses"
 x "I don't know, I haven't thought about it" with no follow-up
 x Lying about a technology on your resume - it WILL come up
 x Checking your phone, or slouching on camera
 x Answering salary first if asked early - deflect politely:
   "I'd like to learn more about the role first; I'm open to a
    discussion based on the full package."

DO:
 O Sit up, smile, make eye contact (or look at the camera)
 O Have your resume, a pen and a notepad ready
 O Say "that's a great question" sparingly - once is fine, five is filler
 O Mirror their language: if they say "backend", you say "backend"
 O Send a thank-you note within 24 hours: reference something specific
   you discussed
```

SAY IT LIKE THIS IN THE INTERVIEW

"Every behavioural answer is a short story with a measurable result, and I never criticise a past employer or manager. My weaknesses are real but paired with what I'm doing about them, and I always have two questions about the team and its practices."

Flashcards

Q. What is the 'Present → Past → Future' structure?
A. The shape for 'tell me about yourself': who you are now, what you've done that's relevant, and what you're looking for next. 90 seconds maximum.

Q. How do you answer 'what's your weakness?'
A. A real, non-fatal weakness plus concrete evidence you're fixing it. Avoid fake humility ('I work too hard') and anything central to the job.

Q. What is STAR?
A. Situation, Task, Action, Result — the structure for every behavioural answer. Own the action with 'I' and put a number in the result.

Q. How should you talk about a previous employer?
A. Neutrally or positively, always. Criticising a past manager or company is a major red flag regardless of whether you're right.

Q. Should you ask questions at the end?
A. Yes, always — two of them, about the team's stack, engineering practices or current challenges. Never about salary in a technical round.

Q. What's the biggest resume mistake?
A. Listing a technology you can't discuss for two minutes. Everything on the page is an invitation to ask about it.

Q. How do you handle a salary question you're not ready for?
A. Deflect politely: 'I'd like to understand the role better first, but I'm open based on the full package' — and research the market range beforehand so you're never guessing.

Quiz

Q1. What's the best structure for 'tell me about a time you faced a challenge'?

[ ] A. Describe the challenge in detail, then stop
[x] B. STAR: Situation, Task, Action, Result — with a number in the result
[ ] C. Explain what your teammate did
[ ] D. Focus on how stressful it was

Why: STAR keeps you concrete and short. Own the action with 'I' and quantify the outcome — that's what makes it credible.

Q2. How should you respond when asked about a previous employer you didn't like?

[ ] A. Explain everything that was wrong there
[x] B. Stay neutral and frame it as a learning experience
[ ] C. Say you can't discuss it
[ ] D. Blame your manager by name

Why: Complaining about past employers is one of the fastest ways to fail an HR round, however justified it feels.

Q3. How long should 'tell me about yourself' be?

[ ] A. 30 seconds
[x] B. About 90 seconds
[ ] C. 5 minutes
[ ] D. As long as they let you talk

Why: Present → Past → Future in ~90 seconds. Long enough to show relevance, short enough to keep their attention and earn follow-up questions.

Q4. Which is a GOOD question to ask the interviewer?

[ ] A. How much vacation do I get?
[x] B. What does the team's stack look like day to day, and how do you handle code reviews?
[ ] C. Did I get the job?
[ ] D. What does your company do?

Why: Ask about the work, the team and the practices. Never ask something you could have found on their website, and save compensation for HR.

5. Final 24 Hours: What To Do & Skip

Plan 15 min

You're four days in. Here's how to spend the last day so you walk in sharp instead of exhausted.

Key points

- **Do**: sleep 7+ hours (memory consolidation is real), re-run the quizzes you got wrong, say answers out loud, skim the cheat sheet, lay out your clothes and ID.
- **Do**: practise writing one full REST controller and one stream pipeline by hand, on paper.
- **Skip**: learning a brand-new framework today. Breadth crammed on day four will not stick, and it'll shake your confidence in what you do know.
- **Skip**: reading 50 new articles. Re-read your own notes instead — retrieval practice beats more input.
- **In the interview**: pause before answering, think out loud, admit gaps honestly, and ask for a hint if you're stuck on code.
- **If you blank on syntax**: say "I'd look up the exact method name, but the approach is..." — that's a perfectly good answer.

Code

The last-24-hours schedule

```
MORNING (3h)
 09:00  Re-run every quiz. Write down the ones you got wrong - TWICE.
 10:30  Flashcard sweep: Day 1 + Day 2 (fast, 30 min)
 11:00  Flashcard sweep: Day 3 + Day 4 (fast, 45 min)
 12:00  Lunch + a walk (yes, seriously)

AFTERNOON (3h)
 13:30  Write a full @RestController by hand on paper, from memory.
 14:00  Write 5 stream pipelines by hand (filter/map/sorted/collect,
        groupingBy, joining, flatMap, reduce).
 14:45  Write a HashMap from scratch? No. But EXPLAIN it out loud.
 15:30  Rehearse your project pitch out loud 3 times. Time it: 45 seconds.
 16:00  Write your two STAR stories.

EVENING (1.5h)
 18:00  Skim the cheat sheet once.
 19:00  Mock interview: have a friend ask you 10 random questions.
 20:00  Pack your bag, lay out clothes, confirm logistics.
 22:00  Sleep. Put the notes down - last-minute cramming hurts recall.

DAY OF
 Wake early, light exercise, breakfast.
 15 min: re-run the output traps only.
 Arrive 10 minutes early (or test your camera/mic the night before).
 First 2 minutes: breathe, smile, and remember they WANT you to be good -
 they wouldn't be interviewing you otherwise.
```

The cheat sheet — last page to read

```
STRINGS     literals pooled; new String() = new object; always .equals()
INTEGER     cache -128..127; never == on wrappers
EXCEPTIONS  checked = compile-time; finally overrides return
COLLECTIONS ArrayList default; HashSet O(1); TreeSet sorted; HashMap 0.75/8
GENERICS    erased at runtime; PECS (producer extends, consumer super)
STREAMS     source -> intermediate (lazy) -> terminal; consumed once
THREADS     start() not run(); sleep holds lock, wait releases it
VOLATILE    visibility yes, atomicity no
SPRING      IoC container; constructor injection; singleton beans
BOOT        auto-config backs off for your beans; starters; profiles
REST        DTOs, @Valid, @RestControllerAdvice, 201 for POST
JPA         Lazy @ManyToOne; JOIN FETCH for N+1; @Transactional on service
SQL         WHERE before GROUP BY, HAVING after; index trade-offs
SECURITY    BCrypt passwords; JWT = signed, readable, stateless
INTERVIEW   3-beat answers; STAR stories; 2 questions of your own

IF YOU BLANK: "I'd look up the exact API, but the approach is..."
That sentence has saved more interviews than any memorised fact.
```

SAY IT LIKE THIS IN THE INTERVIEW

"Four days ago this all felt forgotten. You now have the language back, the framework story straight, and a project pitch that lands. Sleep tonight — recall beats cramming."

Flashcards

Q. What's the single best use of your last day?
A. Retrieval practice: re-run the quizzes and flashcards you got wrong, and say answers out loud. Reading new material makes you feel prepared without actually improving recall.

Q. What should you NOT do on the last day?
A. Start a brand-new topic or framework. It won't stick, and it undermines confidence in what you already know.

Q. What should you hand-write on paper?
A. One full REST controller, five stream pipelines, and your project pitch. Hand-writing exposes the gaps an IDE hides.

Q. What do you say when you blank on syntax?
A. "I'd look up the exact method name, but the approach is..." — then describe the approach. Interviewers care about the reasoning, not the spelling.

Q. Why is sleep actually part of the plan?
A. Memory consolidation happens during sleep. Cramming all night reliably produces worse recall than a rested brain.

Q. What's the fastest credibility-builder in a technical round?
A. Admitting a gap honestly and then reasoning toward an answer. It shows both self-awareness and how you think under pressure.

Quiz

Q1. It's the night before. What's the highest-value activity?

[ ] A. Read 20 new articles on microservices
[x] B. Re-run the quizzes and flashcards you got wrong, and rehearse out loud
[ ] C. Watch a 6-hour Spring tutorial
[ ] D. Re-read all four days from the top

Why: Retrieval practice (testing yourself) beats re-reading by a wide margin. Target your weak spots, then stop and sleep.

Q2. You're given a coding problem and freeze. Best move?

[ ] A. Stay silent and stare at the board
[x] B. Talk through your approach, then ask for a hint if needed
[ ] C. Start typing immediately
[ ] D. Say 'I can't do this one'

Why: Interviewers grade the process: clarify requirements, state a brute-force approach, then optimise. Thinking out loud turns a blank into a conversation.

Q3. You don't remember the exact method name for something. What do you do?

[ ] A. Guess and hope
[x] B. Say you'd look it up, then describe the approach clearly
[ ] C. Skip the question
[ ] D. Pretend it doesn't exist

Why: Nobody codes from memory in real life. Knowing the concept and the approach is what matters — the exact spelling is what the IDE is for.